TP 1 : Arbres de décision

Apprentissage supervisé

1 Préparation des données

Dans l'ensemble des TPs, il vous sera demandé de naviguer dans la documentation des différentes librairies pour obtenir des informations concernant les méthodes, paramètres et descriptions des objets référencés. Ces bibliothèques seront par exemple `NumPy`, `Pandas` ou encore `Scikit-Learn`. Cette dernière est accessible à l'adresse `https://scikit-learn.org/stable/api/index.html`.

Exercice 1.1: Chargez le jeu de données Iris avec la commande suivante :

```
from sklearn.datasets import load_iris
```

Insérez le dans un `DataFrame Pandas` et familiarisez vous avec les méthodes `head` et `info`.

```
iris = load_iris()

dataset = pd.DataFrame(data=..., columns=...)
dataset['species'] = pd.Categorical.from_codes(iris.target, iris.target_names)

print(dataset)
print(dataset.head())
print(dataset.info())
```

Exercice 1.2: Décrivez chacune des variables du jeu de données, avec le type précis, la moyenne, la variance, le min et le max pour les variables quantitatives et le type précis ainsi que l'effectif de chaque valeur pour les variables qualitatives.

```
# Variables quantitatives
print(dataset.describe())

# Variables qualitatives
counts = dataset['species'].value_counts()
print(counts)
```

Exercice 1.3: Affichez l'histogramme de fréquence pour chacune des variables quantitatives du jeu de données. Vous pourrez vous aider de la bibliothèque `matplotlib.pyplot` et de la fonction `hist()`.

```
import matplotlib.pyplot as plt

plt.hist(...)
plt.xlabel(...)
plt.ylabel(...)
plt.title(...)
plt.show()
```

Exercice 1.4: La fonction `pairplot` de `seaborn` permet de représenter les relations deux à deux entre toutes les variables du jeu de données. Affichez le `pairplot` correspondant au jeu de données Iris.

```
import seaborn as sns

sns.pairplot(..., hue=..., diag_kind='kde')
plt.suptitle(...)
plt.show()
```

2 Arbre de décision

Dans cette partie, vous essayerez de prédire l'espèce d'une Iris (Setosa, Versicolor ou Virginica) en fonction de la longueur et la largeur de ses pétales, grâce à un arbre de décision.

Exercice 2.1: Créez un jeu de données à 2 dimensions, en ne retenant que la longueur et la largeur des pétales. Affichez ensuite le nuage de points grâce à la fonction `scatter` de `pyplot`.

```
dataset_petal = dataset.iloc[:, [2,3]]
```

Exercice 2.2: Centrez à présent les données et affichez à nouveau le nuage de points.

Exercice 2.3: Calculez l'inertie totale du nuage de points, sachant que l'inertie mesure la dispersion au centre de gravité :

$$\mathcal{I} = \frac{1}{n}\sum_{i=1}^{n} ||x_i - \bar{x}||^2. \tag{1}$$

Exercice 2.4: (A faire à la fin du TP, pensez à regarder la correction) Montrez que lorsque les données sont centrées, l'inertie peut s'écrire en fonction de la trace du produit matriciel de la transposée de la matrice des données par elle même :

$$\mathcal{I} = \frac{1}{n} Tr(X^T X) = \sum_{j=1}^{p} \sigma_j^2. \tag{2}$$

Exercice 2.5: Calculez le coefficient de corrélation linéaire entre les deux variables étudiées. Vérifiez le résultat obtenu avec la fonction `corrcoef` de `numpy`.
Le coefficient de corrélation linéaire $r_{jj'}$ entre deux variables s'écrit :

$$r_{jj'} = \frac{cov(x_j, x_{j'})}{\sigma_j \sigma'_j}, \tag{3}$$

avec $cov(x_j, x_{j'}) = \frac{1}{n} x_j^T x_{j'}$.

Exercice 2.6: (À faire en fin de TP) Montrez que le coefficient de corrélation linéaire entre deux variables centrées et réduites vaut toujours $\frac{1}{n}\langle x_j, x_{j'} \rangle$.

Exercice 2.7: Rappelons que pour entraîner un arbre de décision, les données n'ont pas besoin d'être centrées et réduites. (Elles seront tout de même centrées pour cet exercice,

et nous utilisons les variables “Petal Width” et “Petal Length”).

Pour commencer, créez une fonction `impurete(p)` qui prend en paramètres un vecteur de probabilités et qui retourne l'indice de Gini associé.
Vérifiez le résultat avec les trois vecteurs suivants :

```
p_test = np.array([
    [0.5, 0.5],
    [0, 1],
    [0.3, 0.7]
])

[print(impurete(p)) for p in p_test]
```

```
0.5
0.0
0.42000000000000004
```

Exercice 2.8: On s'intéresse maintenant au gain associé à la séparation d'un nœud S selon la variable d'indice j au seuil v. Avec la classe suivante :

```
class Noeud:
    def __init__(self, X, y, classes):
        self.X = np.array(X)
        self.y = np.array(y)
        self.n, self.p = self.X.shape
        self.classes = classes
```

`X` représente la matrice des données, de dimension (n, p), `y` représente le vecteur des classes, de dimension n et `classes` regroupe l'ensemble des $K = 3$ classes possibles, soit un vecteur de taille K. Définissez une fonction `gain(S, j, v)` qui retourne le gain associé.
Testez avec $j = 1$ (en informatique, les tableaux commencent généralement à l'indice 0, nous prendrons donc $j = 0$) et $v = -1$:

```
petal_length = np.array(dataset.iloc[:, 2])
petal_width = np.array(dataset.iloc[:, 3])

petal_length = (petal_length - np.mean(petal_length))
petal_width = (petal_width - np.mean(petal_width))

X = np.array([petal_length, petal_width]).T

S = Noeud(X, dataset['species'], np.unique(dataset['species']))
print(gain(S, 0, -1))
```

```
0.3333333333333334
```

On rappelle la formule du gain :

$$Gain(S, j, v) = G(p) - \left[\frac{n^-}{n} G(p^-) + \frac{n^+}{n} G(p^+)\right]. \tag{4}$$

```
def gain(S, j, v):

```

```
    def proportions(S):
        return np.array([np.sum(S.y == classe) / S.n for classe in S.classes])

    n = S.X.shape[0]
    n_gauche = ...
    n_droite = ...

    p = proportions(S)
    p_gauche = proportions(...)
    p_droite = proportions(...)

    return ...
```

Exercice 2.9: Écrivez la fonctions `optimise_gain(S)` qui retourne la variable associée à la meilleure séparation, son seuil et le gain associé. Testez avec le nœud S précédent, La réponse attendue est $(0, -1.31, 0.34)$.

```
def optimise_gain(S):

    ...
    for j in range(S.X.shape[1]):

        valeurs = ...
        for n in range(len(valeurs) - 1):

            v = (valeurs[n] + valeurs[n + 1]) / 2

            gain_actuel = gain(...)
            if gain_actuel > gain_max:
                ...

    return ...

print(optimise_gain(S))
```

Exercice 2.10: Afin de créer l'arbre, l'objet `Noeud` est enrichi comme suit :

```
class Noeud:
    def __init__(self, X, y, classes, j=0, v=0, gauche=None, droite=None):
        self.X = np.array(X)
        self.y = np.array(y)
        self.n, self.p = self.X.shape
        self.classes = classes
        self.j = j
        self.v = v
        self.gauche=gauche
        self.droite=droite
```

Créez une fonction récursive `arbre_recursif(S)` qui retourne la racine de l'arbre construit en ajoutant à chaque séparation, la variable associée $S.j$, le seuil $S.v$ et les nœuds enfants $S.gauche$ et $S.droite$.
Vous pouvez vous aider de :

```
def arbre_recursif(S):

```

```
    S.j, S.v, gain = ...

    if gain == 0:
        return S

    S.gauche = Noeud(S.X[S.X[:, S.j] <= S.v], S.y[S.X[:, S.j] <= S.v], S.classes)
    S.droite = ...

    arbre_recursif(...)
    arbre_recursif(...)

    return S
```

Testez avec le code suivant :

```
def print_tree(S, depth=0):
    indent = "  " * depth
    if S.gauche is None:
        y_list = list(S.y)
        print(f"{indent}Leaf: class={max(set(y_list), key=y_list.count)}")
    else:
        print(f"{indent}Feature[{S.j}] <= {S.v}")
        print_tree(S.gauche, depth+1)
        print_tree(S.droite, depth+1)

print_tree(S)
```

qui devrait retourner :

```
Feature[0] <= -1.3080000000000005
  Leaf: class=setosa
  Feature[1] <= 0.5506666666666664
    Feature[0] <= 1.1919999999999997
      Feature[1] <= 0.45066666666666644
        Leaf: class=versicolor
        Leaf: class=virginica
      Feature[1] <= 0.35066666666666646
        Leaf: class=virginica
        Feature[0] <= 1.6919999999999993
          Leaf: class=versicolor
          Leaf: class=virginica
    Feature[0] <= 1.0919999999999996
      Leaf: class=virginica
      Leaf: class=virginica
```

Exercice 2.11: Écrivez maintenant une fonction `prediction(S, x)` qui prédit la classe de l'instance x avec l'arbre S, testez avec l'ensemble suivant :

```
X_test = [[-2, -0.7], #setosa
          [0.6, -0.1], #versicolor
          [1.7, 1]]   #virginica
```

Exercice 2.12: Afin de pouvoir réutiliser votre arbre sur n'importe quel jeu de données, créez dans un fichier `arbre.py` une classe `MonArbre` qui s'initialise avec un jeu de données (X, y) et possède possède une méthode `predict(self, X_test)` qui retourne la liste des prédictions associées aux instances de `X_test`.
Votre arbre doit pouvoir être appelé comme suit :

```
from arbre import MonArbre

X, y = load_iris(return_X_y = True)

X_test = [[5.5, 4, 1.5, 0.5],
        [5.5, 2, 4.5, 1.5],
        [7.5, 3, 7, 1.5]]

model = MonArbre(X, y)
predictions = model.predict(X_test)

print(predictions)
```

3 La classe DecisionTreeClassifier

Exercice 3.1: La bibliothèque `scikit-learn` propose une classe permettant d'instancier et d'entraîner un arbre de décision.
Pour entraîner le modèle il faudra utiliser la méthode `fit()` comme suit :

```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier()
model.fit(X, y)
```

Vérifiez avec la méthode `predict()` de cette classe, que les prédictions sont les mêmes qu'avec `MonArbre`.

Exercice 3.2: Affichez votre arbre avec le code suivant :

```
from sklearn.tree import plot_tree

plt.figure(figsize=(20,50))
plot_tree(model, feature_names=iris.feature_names, class_names=iris.target_names,
↪  filled=True)
plt.show()
```

Quelles sont les trois feuilles les plus denses ? Quelle est leur indice de Gini ?

Exercice 3.3: Pour testez les performances de notre arbre, nous allons séparer le jeu de données en jeu d'entraînement et jeu de test. La méthode `train_test_split` de `scikit-learn` permet de réaliser cette séparation en précisant la taille du jeu de test souhaité :

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)
```

Entraînez votre modèle sur le jeu d'entrâinement et testez ses performances sur le jeu de test à l'aide de la méthode `score(X_test, y_test)` de la classe `DecisionTreeClassifier`.
Relancez plusieurs fois votre code, que remarquez-vous et pourquoi ? Le score d'exactitude (*accuracy*) est obtenu avec la formule suivante :

$$accuracy = \frac{\text{nombre de prédictions correctes}}{\text{nombre total de prédictions}} \tag{5}$$

Exercice 3.4: Répétez 100 fois aléatoirement l'étape d'entraînement, sur une séparation du jeu de données différente afin d'avoir une moyenne des performances du modèle sur le jeu de données Iris.

Exercice 3.5: La classe `DecisionTreeClassifier` offre aussi plusieurs critères d'élagage pré-entraînement avec les paramètres `max_depths`, qui régule la profondeur maximale de l'arbre lors de sa croissance, et `min_samples_leaf`, qui limite le nombre minimum d'éléments dans une feuille.
Vérifiez si cette régularisation permet d'obtenir de meilleures performances en affichant un graphique de l'évolution des performances moyennes du modèle en fonction de la valeur du paramètre d'élagage. D'abord avec :

```
profondeur_max = [1, 3, 5, 8, 10, 12, 15]
```

Puis avec :

```
densite_min = [1, 2, 3, 4, 5, 6, 7]
```

Exercice 3.6: Pour étudier le comportement de cet élagage, répétez la dernière étape sur un autre jeu de données plus complexe et vérifiez l'effet de la régularisation.

```
from sklearn.datasets import load_breast_cancer
X, y = load_breast_cancer(return_X_y = True)
```

4 MonArbre élagué

Exercice 4.1: Implémentez l'élagage par pronfodeur maximale et par densité de feuille minimale dans la classe `MonArbre`. Rendez également compatible le modèle avec les variables qualitatives et avec la régression.